Cahier des charges – Plateforme d'apprentissage d'un langage de programmation par niveaux (Java Swing)

Nov 3, 2026 · @Bader

Les diagrammes (cas d'utilisation, classes, états, activités, navigation) se trouvent dans le document Doc.

1. Contexte et présentation

Le projet consiste à concevoir l'interface d'une plateforme d'apprentissage d'un langage de programmation, organisée en niveaux de difficulté, du niveau débutant au niveau avancé, avec une progression pédagogique contrôlée automatiquement.

Chaque niveau regroupe des cours, des exemples, des exercices pratiques et un test de validation. L'apprenant ne peut accéder au niveau suivant que s'il satisfait aux conditions de validation fixées par l'enseignant. Le système compare le résultat du test au seuil de réussite, applique les règles de passage et met à jour la progression de chaque apprenant.

Ce projet est réalisé dans le cadre du module d'IHM (Interface Homme-Machine). Il porte uniquement sur le front-end, développé en Java Swing.

2. Objectifs

L'objectif principal est de livrer une interface Swing cohérente et ergonomique qui permet d'apprendre pas à pas, en laissant à l'enseignant la maîtrise de la stratégie pédagogique.

- Offrir un parcours d'apprentissage structuré et progressif, organisé en niveaux.
- Permettre à l'enseignant de créer et de gérer les cours, les exercices et les tests.
- Automatiser la validation des acquis et le déblocage du niveau suivant.
- Permettre à l'apprenant de consulter ses résultats, son niveau actuel et les contenus accessibles.
- Permettre à l'enseignant de suivre les performances individuelles et collectives.
- Permettre à l'administrateur de gérer les comptes et les rôles.
- Respecter les principes d'ergonomie d'une IHM : clarté, cohérence, retour d'information, prévention des erreurs.

3. Périmètre et contraintes

Le projet couvre uniquement la couche présentation : les écrans, la navigation et la logique de progression exécutées localement sur des données simulées.

Inclus dans le périmètre

- Toutes les interfaces des trois rôles (administrateur, enseignant, apprenant).
- La logique de validation des niveaux (comparaison du score au seuil, déblocage, nouvelles tentatives).
- Des données de démonstration chargées en mémoire ou depuis un fichier local (JSON, CSV ou texte).

Hors périmètre

- Serveur, API réseau et base de données.
- Exécution et compilation réelles du code des apprenants (les exercices sont corrigés par comparaison de réponses ou par QCM).
- Sécurité avancée (chiffrement, sessions distantes).

Contraintes techniques

- Langage : Java (JDK 17 ou supérieur recommandé).
- Bibliothèque graphique : Swing (`javax.swing`), avec un look and feel unifié (Nimbus ou FlatLaf si autorisé).
- Architecture : MVC, avec séparation entre le modèle, les vues et les contrôleurs.
- Application exécutable sur poste de bureau, sans connexion internet.

4. Acteurs du système

Trois rôles utilisent la plateforme ; chacun voit uniquement les écrans correspondant à ses droits.

Acteur	Rôle	Actions principales
Administrateur	Gère la plateforme	Gérer les comptes, attribuer les rôles et les droits d'accès, superviser la plateforme
Enseignant	Organise le parcours pédagogique	Créer, modifier et supprimer cours, exercices et tests ; organiser les niveaux et leur période ; définir seuils et conditions de passage ; suivre les résultats et la progression

Acteur	Rôle	Actions principales
Apprenant	Apprend et progresse	Consulter les cours de son niveau, réaliser les exercices, passer les tests, consulter notes et progression, repasser un test selon les règles

5. Besoins fonctionnels

Les exigences sont regroupées par module ; la priorité indique l'ordre de développement (Haute = indispensable à la démonstration).

ID	Module	Exigence	Acteur	Priorité
F01	Utilisateurs	Authentification (identifiant et mot de passe) et déconnexion	Tous	Haute
F02	Utilisateurs	Inscription d'un apprenant et gestion du profil	Apprenant	Moyenne
F03	Utilisateurs	Création, modification, suppression et recherche de comptes ; attribution des rôles	Administrateur	Haute
F04	Utilisateurs	Affichage des écrans et des actions selon le rôle	Tous	Haute
F05	Contenus	Créer, modifier et supprimer un cours	Enseignant	Haute
F06	Contenus	Créer des niveaux, définir leur ordre et leur période de formation	Enseignant	Haute
F07	Contenus	Associer cours, exercices et test à un niveau	Enseignant	Haute
F08	Contenus	Ajouter des exemples de code et des ressources complémentaires	Enseignant	Moyenne
F09	Contenus	Réordonner les contenus d'un niveau	Enseignant	Basse
F10	Évaluations	Créer des exercices et des tests (questions, réponses attendues, barème)	Enseignant	Haute
F11	Évaluations	Définir le seuil de réussite, le nombre de tentatives et les conditions de passage	Enseignant	Haute

ID	Module	Exigence	Acteur	Priorité
F12	Évaluations	Correction automatique des QCM et des réponses courtes	Système	Haute
F13	Évaluations	Enregistrer les notes et les tentatives	Système	Haute
F14	Progression	Vérifier les conditions et valider ou non le niveau courant	Système	Haute
F15	Progression	Débloquer automatiquement le niveau suivant	Système	Haute
F16	Progression	Verrouiller l'accès aux niveaux non validés	Système	Haute
F17	Progression	Gérer les nouvelles tentatives selon les règles définies	Système	Moyenne
F18	Apprenant	Consulter les cours de son niveau, les exemples et les ressources	Apprenant	Haute
F19	Apprenant	Réaliser les exercices et passer le test de validation	Apprenant	Haute
F20	Apprenant	Afficher le niveau actuel, les niveaux validés et ceux à débloquer	Apprenant	Haute
F21	Apprenant	Consulter l'historique des tests, les notes et les taux de réussite	Apprenant	Moyenne
F22	Résultats	Afficher immédiatement le résultat et la notification de passage au niveau suivant	Système	Haute
F23	Résultats	Indiquer les conditions non satisfaites et les contenus à revoir en cas d'échec	Système	Moyenne
F24	Suivi	Tableau de suivi des apprenants (niveau, notes, taux de réussite) avec filtres	Enseignant	Moyenne
F25	Suivi	Statistiques collectives par niveau	Enseignant	Basse

6. Règles de gestion de la progression

Un niveau est validé uniquement si les conditions définies par l'enseignant sont satisfaites ; sinon le niveau suivant reste verrouillé. Les valeurs ci-dessous sont des paramètres par défaut, modifiables par l'enseignant.

ID	Règle
R1	L'apprenant commence au niveau 1 ; seul ce niveau est déverrouillé à l'inscription.
R2	Chaque test est associé à un seul niveau et possède un barème et un seuil de réussite (par défaut 60 %).
R3	La note est convertie en pourcentage : note obtenue ÷ note maximale.
R4	Le niveau est validé si le pourcentage est supérieur ou égal au seuil. L'enseignant peut ajouter des conditions plus strictes (à confirmer : seuil plus élevé, exercices obligatoires terminés).
R5	Si le niveau est validé, le niveau suivant est débloqué et une notification s'affiche.
R6	Si le niveau n'est pas validé, le niveau suivant reste verrouillé et l'apprenant voit la note, le seuil et l'écart à combler.
R7	Le nombre de tentatives est limité (par défaut 3, à confirmer) ; la meilleure note est conservée.
R8	Chaque tentative est enregistrée dans l'historique (date, note, résultat).
R9	Un niveau validé reste consultable ; seul le dernier niveau validé met fin au parcours.

7. Écrans de l'application

L'application compte 16 écrans ; la fenêtre principale (`JFrame`) en affiche un à la fois avec un `CardLayout` , et le rôle choisi à la connexion détermine le tableau de bord.

Rôle	Écran	Contenu principal
Tous	Connexion	Identifiant, mot de passe, accès à l'inscription
Apprenant	Inscription	Création d'un compte apprenant
Apprenant	Tableau de bord	Niveau actuel, niveaux verrouillés, déverrouillés et validés, barre de progression
Apprenant	Cours	Contenu du cours et exemples de code
Apprenant	Exercices	Énoncé, zone de réponse, correction
Apprenant	Test	Questions du test de validation du niveau
Apprenant	Résultats	Note, seuil, message de passage ou d'échec, contenus à revoir
Apprenant	Progression et historique	Tentatives, notes, taux de réussite
Enseignant	Tableau de bord	Accès aux écrans de gestion et de suivi
Enseignant	Niveaux et cours	Création, modification, suppression, ordre et période
Enseignant	Exercices et tests	Questions, réponses attendues, barème
Enseignant	Seuils et règles	Seuil de réussite, tentatives, conditions de passage
Enseignant	Suivi des apprenants	Tableau des niveaux, notes et taux de réussite
Administrateur	Tableau de bord	Accès à la gestion des comptes
Administrateur	Gestion des comptes	Création, modification, suppression, recherche
Administrateur	Attribution des rôles	Rôles et droits d'accès

8. Besoins non fonctionnels et critères d'ergonomie

L'interface doit être jugée sur son ergonomie autant que sur ses fonctions ; les critères ci-dessous servent de grille de vérification.

Critère	Exigence pour l'application Swing
Cohérence	Même disposition, mêmes libellés et mêmes icônes sur tous les écrans ; un seul look and feel.

Critère	Exigence pour l'application Swing
Visibilité de l'état	Barre de progression, badge « verrouillé / déverrouillé / validé » sur chaque niveau, niveau actuel toujours affiché.
Retour d'information	Message clair après chaque action (résultat du test, niveau débloqué, enregistrement réussi).
Prévention des erreurs	Champs validés avant envoi, confirmation avant toute suppression, boutons désactivés si l'action est interdite.
Aide à la compréhension	En cas d'échec, afficher le seuil, la note obtenue et les contenus à revoir.
Navigation	Accès à n'importe quel écran en trois clics maximum ; bouton de retour et fil d'Ariane.
Lisibilité	Police lisible, contrastes suffisants, zone de code en police à chasse fixe.
Accessibilité clavier	Raccourcis clavier, ordre de tabulation logique, touche Entrée pour valider.
Redimensionnement	Fenêtre redimensionnable avec des gestionnaires de disposition (pas de positions absolues).
Performance	Ouverture des écrans sans blocage visible ; chargement des données dans un thread séparé si nécessaire (`SwingWorker`).
Maintenabilité	Code organisé en packages MVC, classes documentées, données de démonstration séparées du code.

9. Architecture technique

L'application suit le patron MVC : les vues Swing n'appliquent aucune règle métier, elles délèguent aux contrôleurs, qui appellent le modèle. La règle de déblocage des niveaux est isolée dans un seul service, facile à tester.

Package	Contenu	Composants Swing / remarques
`model`	Utilisateur (Administrateur, Enseignant, Apprenant), Niveau, Cours, Exercice, Test, Question, Tentative, Progression	Classes Java simples (POJO)
`service`	`ProgressionService` (validation et déblocage), `CorrectionService` (correction automatique), `AuthService`	Contient les règles R1 à R9
`data`	Dépôts de données simulées (`DataStore`) ; lecture et écriture d'un fichier local JSON ou CSV	Aucune base de données
`controller`	`LoginController` , `LevelController` , `TestController` , `TeacherController` , `AdminController`	Écoutent les événements (`ActionListener`)
`view`	`MainFrame` , panneaux de connexion, tableau de bord, cours, test, résultats, éditeur enseignant, gestion des utilisateurs	`JFrame` , `JPanel` , `CardLayout` , `JTabbedPane` , `JTable` , `JProgressBar` , `JTextArea` , `JOptionPane`
`util`	Constantes, styles, icônes, validation de saisie	Look and feel et thème communs

Les données de démonstration comprennent au minimum un administrateur, un enseignant, deux apprenants à des niveaux différents et trois niveaux de cours, afin de montrer les cas validé, échoué et verrouillé.

10. Livrables, planning et points à clarifier

Le projet se déroule en cinq phases ; les dates restent à fixer avec l'enseignant.

Phase	Contenu	Livrable
1. Cadrage	Analyse du sujet, acteurs, cas d'utilisation, questions à l'enseignant	Ce cahier des charges, cas d'utilisation
2. Conception	Diagrammes de classes, d'états et d'activités, maquettes, carte de navigation	Dossier de conception, maquettes

Phase	Contenu	Livrable
3. Prototype	Squelette Swing : connexion, tableau de bord, niveaux verrouillés, `ProgressionService`	Premier prototype exécutable
4. Développement	Écrans apprenant, enseignant et administrateur, correction automatique, données simulées	Application complète
5. Tests et rendu	Scénarios de test, correction des défauts, rapport, démonstration	Code source, rapport, démonstration

Points à clarifier avec l'enseignant

- [ ] Quel langage de programmation la plateforme enseigne-t-elle ?
- [ ] Les données peuvent-elles être simulées, ou faut-il les conserver dans un fichier ?
- [ ] La connexion doit-elle être réelle ou simplement un choix de rôle pour la démonstration ?
- [ ] La correction des exercices de programmation est-elle attendue, ou les QCM et réponses courtes suffisent-ils ?
- [ ] Que signifie la mention « seuils + dur » dans la section sur les conditions de passage ?
- [ ] Combien de tentatives par niveau, et y a-t-il un délai d'attente entre deux tentatives ?
- [ ] Quelles bibliothèques sont autorisées (FlatLaf, MigLayout, RSyntaxTextArea) ?
- [ ] Quels sont les livrables, les dates de rendu, et le travail est-il individuel ou en groupe ?